

21720 Programació concurrent

1. Introducció a la programació concurrent

Conceptes previs

- Els programes actuals són inherentment concurrents o distribuïts:
 - Interfícies gràfiques basades en esdeveniments
 - Sistemes de temps real
 - Aplicacions d'Internet com jocs multi-usuari
- Independència de la tecnologia:
 - Conceptes: Intercalat, exclusió mútua i seguretat.
 - Eines: Semàfors, monitors, canals i missatges.
 - Problemes: Regió crítica, productor-consumidor, lectors i escriptors i consens



Aleksandr Ilyushin

@chronum

Segueix

Multithreaded programming - theory and practice.



RETUITS
7.336

AGRADA A
3.561



1:30 - 4 des. 2014



7,3m



3,6m



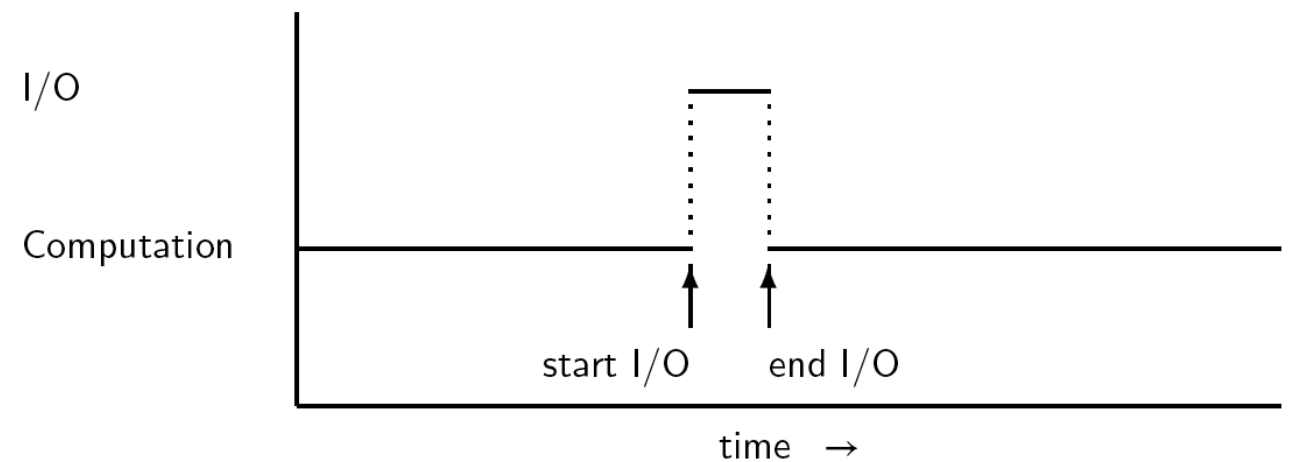
Concurrencia, definicions bàsiques

- *Programa concurrent*: Conjunt de programes seqüencials que poden ser executats en paral·lel
- *Processos*: Programes seqüencials que comprenen un programa concurrent
- *Paral·lel*: Sistema en que les execucions de diferents programes es superposen en el temps executant-se en processadors separats. Relacionats o no
- *Concurrencia*: Maneig de la relació entre processos. Paral·lelisme potencial.
- **ESTRUCTURA** vs **EXECUCIÓ**
- <http://blog.golang.org/concurrency-is-not-parallelism>
- <http://stackoverflow.com/questions/1897993/difference-between-concurrent-programming-and-parallel-programming> !!!!!!!

- **ESTRUCTURA** vs **EXECUCIÓ**
- **Concurrència**: tractar elements al mateix temps
- **Parel·lelisme**: executar elements al mateix temps (poden estar relacionats o no)

Concurrencia, definicions bàsiques

- *Multitasca* (SO):
Generalització del concepte de superposició de l'E/S amb el còmput. *Scheduler* implementant talls de temps per al compartiment de recursos
- *Multitasca* (lleng. prog.):
Multithreading. Amb els fils es poden programar operacions concurrents/paral·leles en un sol programa



Terminologia

- **Procés** / *fil (thread)*: Concurrència / lleng. programació. Terme popularitzat pels *pthread*s POSIX especialment dels sistemes UNIX. Terme anàleg amb subtils diferències:
 - Un procés s'executa en el seu Espai d'adreces
 - Un fil s'executa a l'espai d'adreces d'un procés (fil principal de Java)
- **Procés** / *tasques (task)* de Ada. Concepte equivalent excepte quan s'usa en sistemes de temps real per denotar petites unitats de treball a sistemes de temps real

Múltiples ordinadors

- *Multiprocessadors* per al comput p.e. per a tasques de simulació
- *Multiprocessadors* dins un ordinador, processadors gràfics, processadors específics d'E/S i comunicacions
- *Sistemes distribuïts* p.e. granges de servidors

Sincronització

- Programació concurrent: Sincronització de diferents processos perquè es puguin **comunicar**
- Eficiència i Seguretat

Concurrencia a Java

- Suporta la concurrència mitjançant el objectes del tipus *Thread*. Es poden escriure classes com a extensió de *Thread*. Java no té herència múltiple per això és més recomanable implementar la interfície *Runnable*
- És necessari definir un mètode *run* amb el codi que ha d'executar cada procés.
- Quan una classe s'ha definit així es poden crear atributs d'aquest tipus invocant al constructor de la classe.
- Per executar el procés s'ha d'invocar el mètode *start*. Amb el mètode *join* s'empra per esperar a la terminació dels processos. La invocació d'aquest mètodes requereix la gestió de l'excepció *InterruptedException*.
- Les variables *static* seran compartides per tots els objectes de la classe
- Una variable es pot declarar *volatile*. Tots els tipus primitius excepte *long* i *double* són atòmics, i aquests també si es declaren *volatile*.

- Threads de Java: <https://docs.oracle.com/javase/7/docs/api/java/lang/Thread.html>

Threads.java

```
public class Threads implements Runnable {
    static final int THREADS = 2;
    int id;

    public Threads(int id) {
        this.id = id;
    }

    @Override
    public void run() {
        System.out.printf("Hi, I'm thread %d\n", id);
    }

    public static void main(String[] args) throws InterruptedException {
        Thread[] threads = new Thread[THREADS];
        int i;

        for (i=0; i< THREADS; i++) {
            threads[i] = new Thread(new Threads(i));
            threads[i].start();
        }

        for (i=0; i< THREADS; i++) {
            threads[i].join();
        }
    }
}
```

Concurrencia a *Python*

- <http://www.dabeaz.com/usenix2009/concurrent/>
- <https://docs.python.org/3/library/concurrency.html>
- S'hereta des de la classe *threading* i s'ha de redefinir el mètode *run()*
- Per a llançar les processos es creen els objectes i es crida al mètode *start()*. Els *threads* s'executen fins que s'atura el mètode *run()*
- Una altra manera de llançar els fils és crear l'objecte però el mètode *run* crida a la funció
- Quan un fil comença s'executa de forma independent. El mètode *join()* s'usa per esperar que el fil acabi
- <https://stackoverflow.com/questions/3044580/multiprocessing-vs-threading-python>

- Threads a Python

threads.py

```
import threading

THREADS = 4

def thread():
    print("Hi, I'm the thread {}".format(threading.current_thread().name))

def main():
    threads = []

    for i in range(THREADS):
        # Create new threads
        t = threading.Thread(target=thread)
        threads.append(t)
        t.start() # start the thread

    # Wait for all threads to complete
    for t in threads:
        t.join()

    print("End")

if __name__ == "__main__":
    main()
```

Global Interpreter Lock GIL de *Python*

- Bytecode: codi intermedi generat a partir del codi font Python que executa l'interpret de CPython.
- **GIL**: És un lock global que controla l'execució del bytecode Python.
- **Només un thread** pot executar bytecode Python alhora dins d'un procés CPython amb el GIL.
- Els threads poden ser **concurrents**, però no poden obtenir **paral·lelisme** real per a codi Python CPU-bound
- **No impedeix la concurrència**: els threads s'executen de manera intercalada
- En tasques CPU-bound: augmentar el nombre de threads no sol accelerar l'execució; fins i tot pot empitjorar el temps per l'overhead de gestió dels threads i del GIL.
- <http://www.dabeaz.com/python/GIL.pdf>
- <https://www.youtube.com/watch?v=KVKufdTphKs>

Global Interpreter Lock GIL de *Python*

- **Python 3.13 (2024)**: primera versió estable de CPython que incorpora una compilació free-threaded experimental, capaç d'executar-se sense GIL.
- Possibilita **parallelisme real** de codi Python en diversos nuclis.
- Eliminar el GIL no implica **speedup lineal**; el rendiment continua limitat pel nombre de nuclis, sincronització, memòria i overhead.
- https://docs.python.org/3/howto/free-threading-python.html?utm_source=chatgpt.com

Global Interpreter Lock GIL de *Python*

- Resultat del codi CPU-bound que fa 100_000_000 iteracions calculant la suma de quadrats dels nombre d'un interval. Executats amb un MacBook Pro amb Apple M1 de 8 nuclis i 16 GB

Fils	Python 3.9 — GIL	Python 3.14t — sense GIL
1	7,813 s	5,861 s
2	12,409 s	3,169 s
4	12,196 s	2,484 s
8	12,918 s	2,195 s

Concurrencia a C

- <https://computing.llnl.gov/tutorials/pthreads/>
- Les llibreries *POSIX thread* suporten la concurrència per C/C++
- Les operacions de *pthread* inclouen creació, acabament, sincronització (*join*, *blocking*), *sheduling*, gestió de dades i interacció de processos
- Els *threads* d'un mateix procés comparteixen: instruccions del procés, dades, fitxers oberts, senyals, directori de treball, identificador d'usuari i grup
- Cada thread té un únic: Thread ID, conjunt de registres, apuntador de pila, pila de variables locals, adreces de retorn, màscara de senyal, prioritat i valor de retorn

- Threads a C:

```
#define NUM_THREADS 4

// Just used to send the index of the id
struct tdata {
    int tid;
};

int counter = 0;

void *count(void *ptr) {
    int tid = ((struct tdata *) ptr)->tid;
    printf("Hi, I'm thread %d\n", tid);
    pthread_exit(NULL);
}

int main (int argc, char *argv[]) {
    pthread_t threads[NUM_THREADS];
    int rc, i;
    struct tdata id[NUM_THREADS];
    for(i=0; i<NUM_THREADS; i++){
        id[i].tid = i;
        rc = pthread_create(&threads[i], NULL, count, (void *) &id[i]);
    }
    for(i=0; i<NUM_THREADS; i++){
        pthread_join(threads[i], NULL);
    }

    pthread_exit(NULL);
}
```

Concurrencia a Go

- <https://tour.golang.org/concurrency/1>
- *Goroutines*: Execucions independents de funcions llançades per una sentència go
- Tenen les seves pròpies cridades a la pila. Es poden tenir moltes *goroutines* (centenes de milers)
- NO són un *thread* de Go. Un *thread* per programa amb centenars de *goroutines*

- Goroutines en Go

```
package main

import (
    "fmt"
)

const (
    Goroutines = 2
)

func run(id int, done chan bool) {
    fmt.Printf("Hi, I'm thread %d\n", id)
    done <- true
}

func main() {
    done := make(chan bool, 1)
    for i := 0; i < Goroutines; i++ {
        go run(i, done)
    }
    for i := 0; i < Goroutines; i++ {
        <-done
    }
}
```

Concurrencia a Ada

- El terme d'Ada per a procés és *task*. Que es declara com una especificació i un *body*.
- La tasca s'activa amb el seu *begin*. Un subprograma no acaba fins que ho facin les tasques que ha activat.
- Una variable podrà ser declarada volàtil o atòmica usant els *pragma Volatile* o *Atomic*. Els *pragma Volatile_Components* i *Atomic_Components* s'usen per fer els arrays volàtils o atòmics.

- Tasques en Ada

```
with Ada.Text_IO; use Ada.Text_IO;
```

```
procedure Tasques3 is
```

```
    task type Worker (Id : Positive);
```

```
    task body Worker is  
    begin
```

```
        ⚡ Put_Line ("Hi I'm the thread" & Positive'Image (Id));  
    end Worker;
```

```
    W1 : Worker (Id => 1);
```

```
    W2 : Worker (Id => 2);
```

```
    W3 : Worker (Id => 3);
```

```
    W4 : Worker (Id => 4);
```

```
begin
```

```
    null; -- El programa espera automàticament que totes les tasques acabin  
end Tasques3;
```

- Tasques en Ada

```
procedure Tasques is

    task type tasca is
        entry Start (Idx: in integer);
    end tasca;

    task body tasca is
        My_Idx : integer;
    begin
        accept Start (Idx: in Integer) do
            My_Idx := Idx;
        end Start;
        Put_Line ("Hi, i'm task " & My_Idx'img);
    end tasca;

    THREADS : constant integer := 4;
    type tasques is array (1..THREADS) of tasca;
    t: tasques;

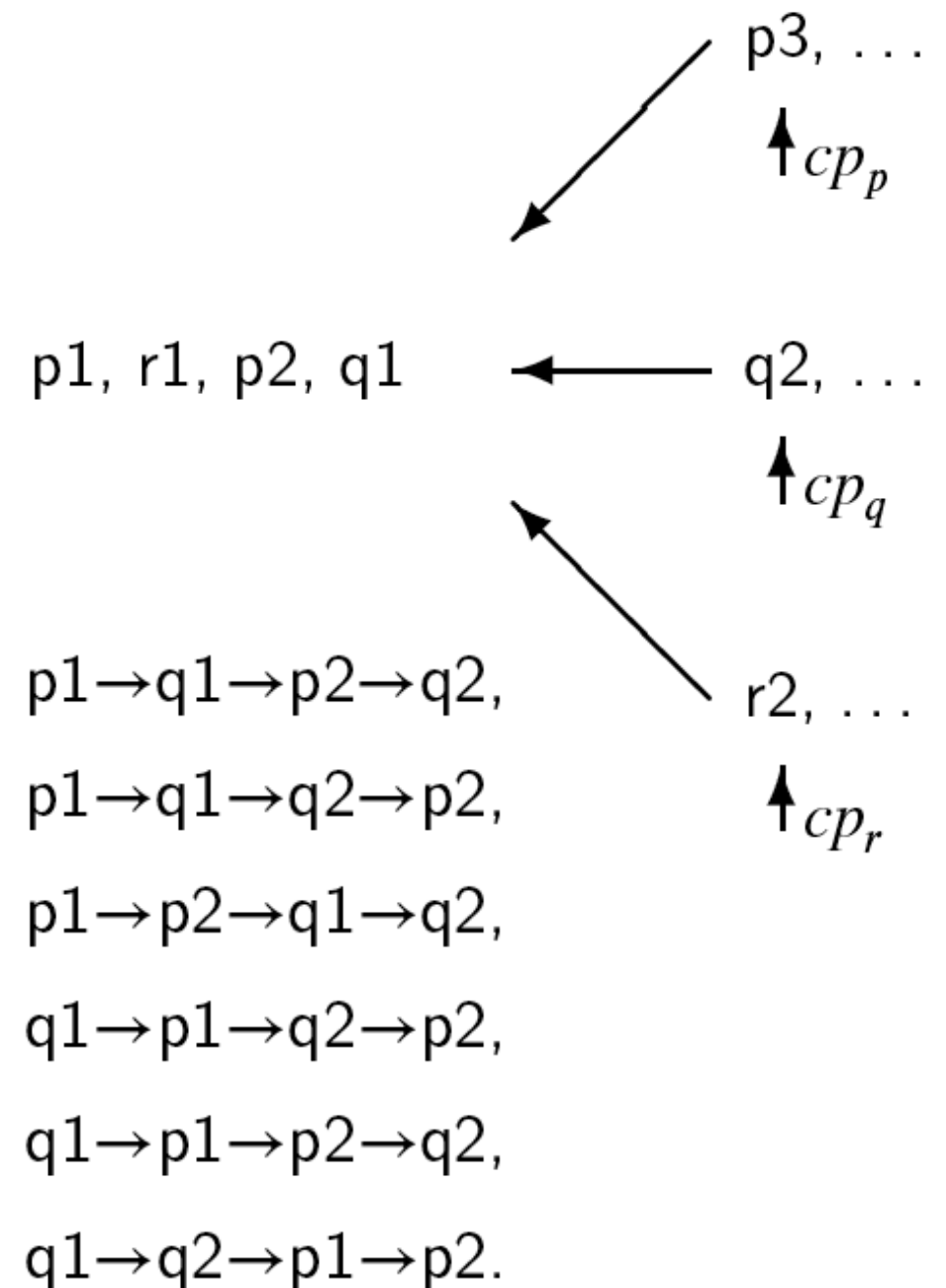
begin
    for Idx in 1..THREADS loop
        t(Idx).Start(Idx);
    end loop;
end Tasques;
```

Abstracció

- Encapsulació: Un mòdul de programa es divideix en especificacions públiques i implementacions amagades
- Concurrència: És una abstracció que s'ha dissenyat perquè sigui possible raonar sobre el comportament dinàmic dels programes

Definicions: Execució concurrent com intercalat de sentències atòmiques

- Un *programa concurrent* consisteix en un conjunt finit de processos seqüencials.
- Els processos estan escrits usant un conjunt finit de *sentències atòmiques*.
- L'execució d'un programa concurrent implica l'execució d'una seqüència de sentències atòmiques obtingudes per la *intercalació arbitrària* de sentències atòmiques dels processos.



Definicions: Execució concurrent com intercalat de declaracions atòmiques (cont.)

- Un *càlcul* o *escenari* és l'execució d'una seqüència que ocorre com a resultat de l'intercalat
- Durant un càlcul l'*apuntador de control* d'un procés indica quina és la següent sentència que serà executada per quin procés. Cada procés té el seu apuntador de control

Algorithm 2.1: Trivial concurrent program	
integer $n \leftarrow 0$	
p	q
integer $k1 \leftarrow 1$ p1: $n \leftarrow k1$	integer $k2 \leftarrow 2$ q1: $n \leftarrow k2$

Definicions: Execució concurrent com intercalat de declaracions atòmiques (cont.)

Algorithm 2.2: Trivial sequential program

integer $n \leftarrow 0$

integer $k1 \leftarrow 1$

integer $k2 \leftarrow 2$

p1: $n \leftarrow k1$

p2: $n \leftarrow k2$

- L'*estat* d'un algorisme concurrent és una tupla consistent en un element per a cada procés que és una etiqueta i un element per a cada variable global o local

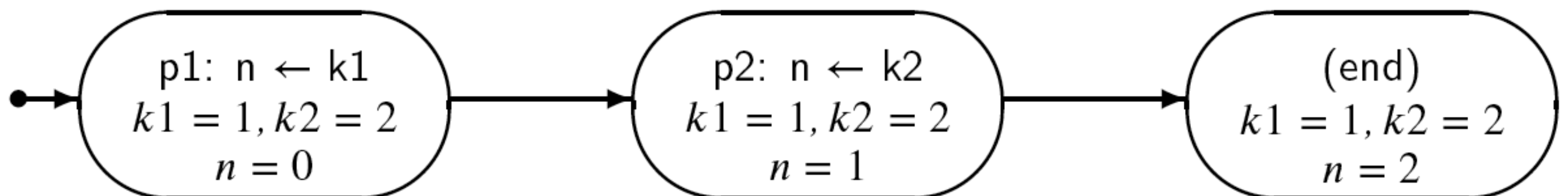
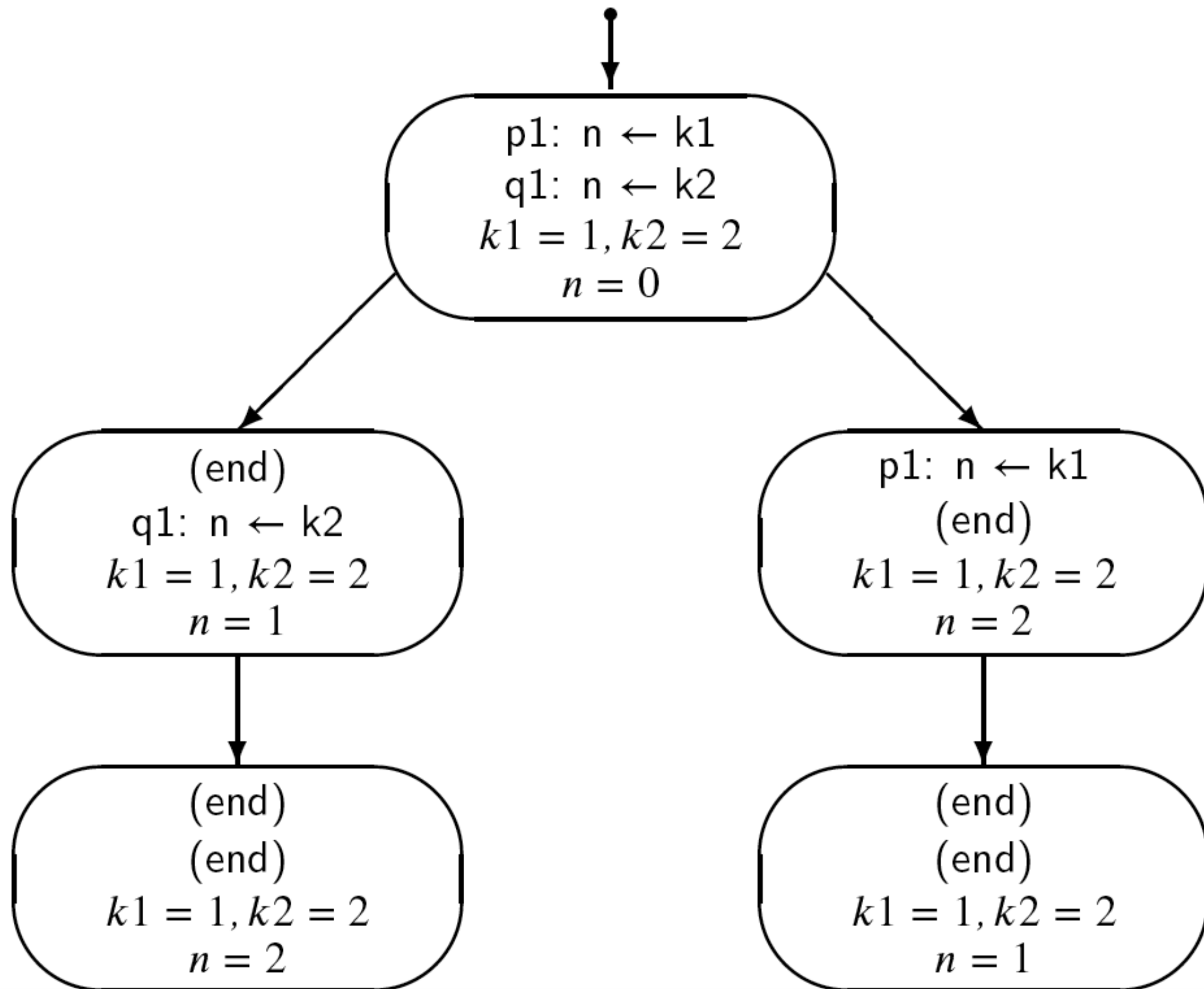


Diagrama d'estats

Definicions: Execució concurrent com intercalat de declaracions atòmiques (cont.)

- Siguin s_1 i s_2 *estats*. Hi ha una *transició* entre s_1 i s_2 si executant una sentència en s_1 es canvia l'estat a s_2 . La sentència executada ha de ser una de les apuntades per l'apuntador de control.
- Un *diagrama d'estats* es defineix inductivament. L'estat inicial conté un node simple. Per a cada estat només hi ha un node etiquetat amb aquest estat. El conjunt d'*estats accessibles* és el conjunt d'estats al diagrama d'estats.

Diagrama d'estats per al programa concurrent



Escenari corresponent al camí de l'esquerra
de l'algorisme anterior

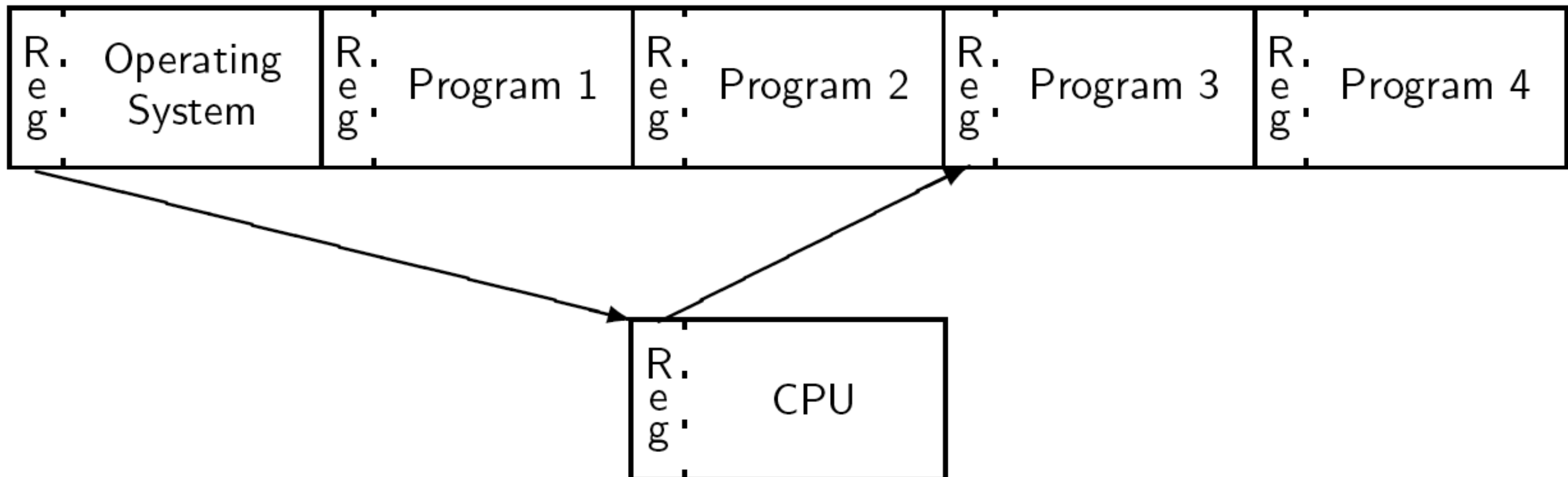
Process p	Process q	n	k1	k2
p1: $n \leftarrow k1$	q1: $n \leftarrow k2$	0	1	2
(end)	q1: $n \leftarrow k2$	1	1	2
(end)	(end)	2	1	2

Abstracció en concurrència

- S'assumeix que es té una vista de l'estat global del sistema i que la sentència d'un dels processos s'executa i es completa antes que l'execució d'una sentència d'un altre procés comenci.
- Es considera que l'execució d'un programa concurrent és duta a terme per una entitat global que a cada pas selecciona de forma arbitraria el procés des del que la següent sentència serà executada. D'aquí prové el terme d'intercalat.

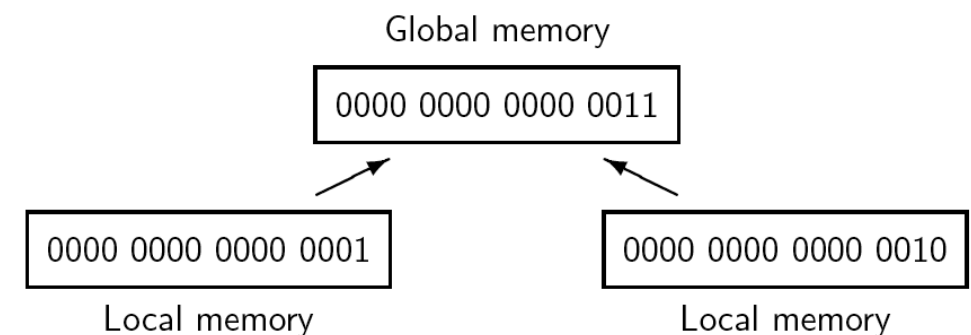
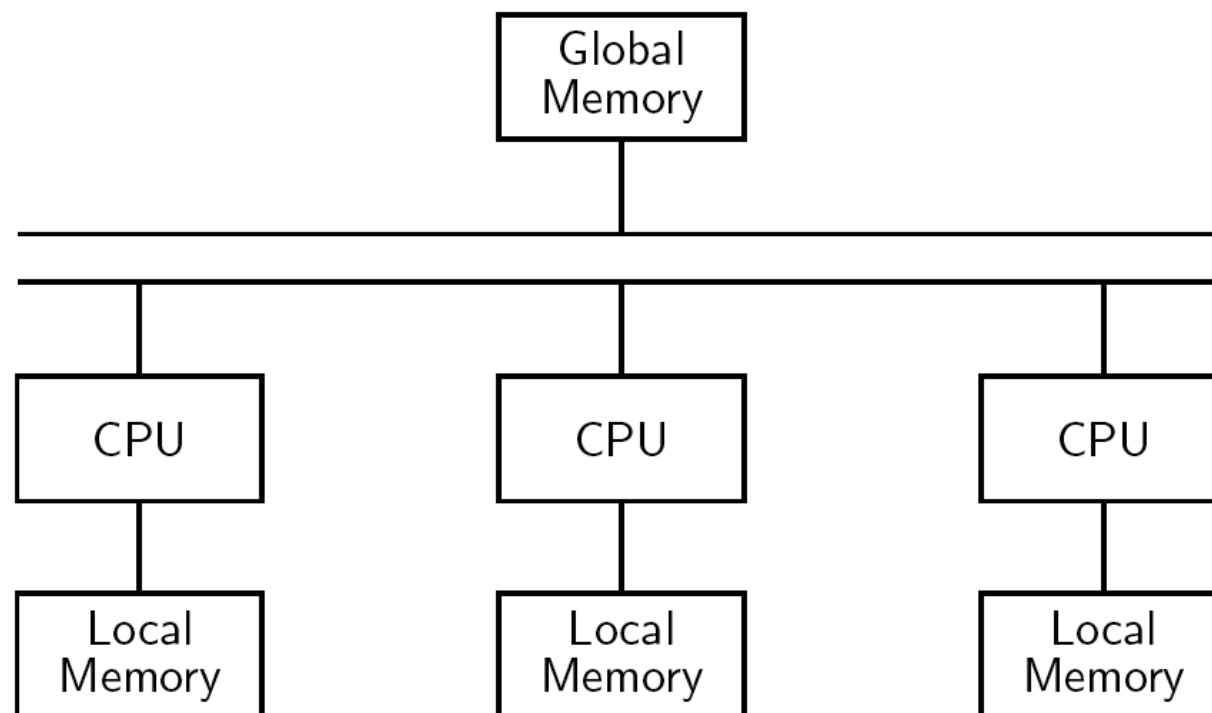
Abstracció en sistemes multitasca

- La selecció de la següent instrucció es duta a terme per la UCP i el SO. Maneig d'interrupcions, planificador, canvi de context



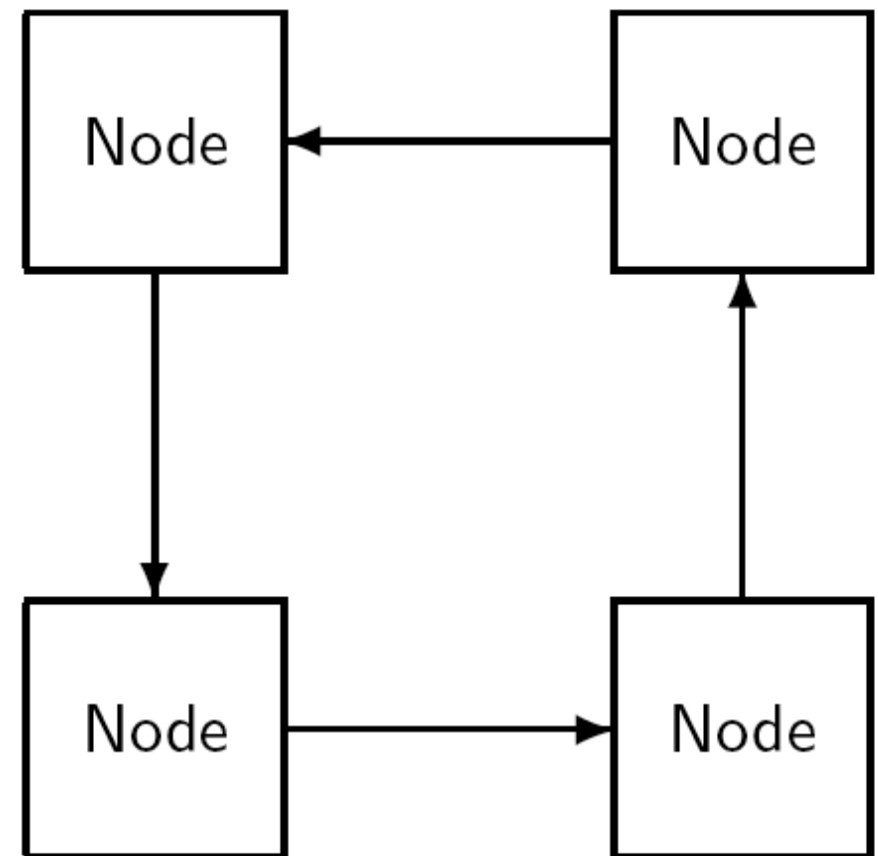
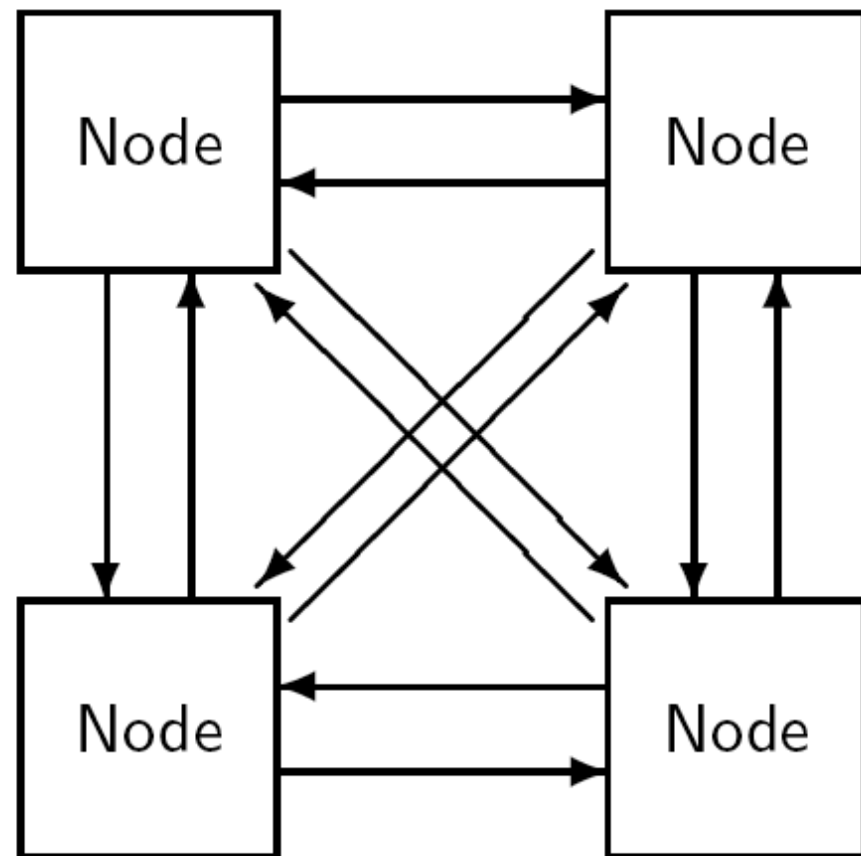
Abstracció en ordinadors multiprocessador

- Si es tenen suficients CPUs no hi ha problema d'intercalat
- Si més d'una CPU intenta escriure a la mateixa cel·la de memòria local hi ha un problema potencial.
- A la pràctica això no passa per que el *hardware* de la memòria està dissenyat per això. L'atomicitat i l'intercalat són duits a terme pel *hardware*



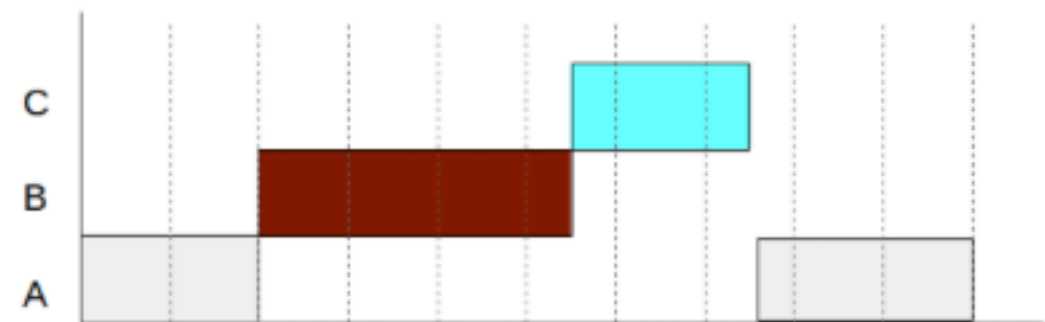
Abstracció en sistemes distribuïts

- Cada node executa les seves pròpies sentències, envia o rep missatges. L'intercalat s'entén com la seqüència de les sentències i com el requeriment de que el missatge ha de ser enviant abans de ser rebut
- Diferències entre distribuïts i concurrents: A multitasca i multiprocés la memòria global és accessible a tots els processos, aquí depèn de la topologia del sistema i de la tolerància a fallades

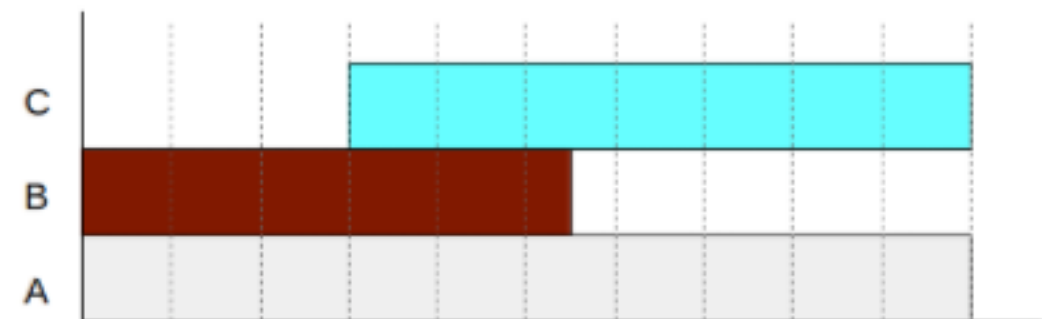


Intercalat arbitrari

- Després de l'execució de cada sentència, la següent sentència pot venir de qualsevol altre procés
- S'ha d'assegurar que es tenen un nombre finit de sentències i no els intervals de temps entre elles. La correctesa no depèn del temps d'execució.
- És difícil, si no impossible, repetir precisament l'execució d'un programa concurrent



Intercalat exclusiu



Multiprocés

Sentències atòmiques

- Aquella que s'executa fins al seu acabament sense intercalat d'un altre procés

Algorithm 2.3: Atomic assignment statements

integer $n \leftarrow 0$

p

p1: $n \leftarrow n + 1$

q

q1: $n \leftarrow n + 1$

Process p	Process q	n
p1: $n \leftarrow n + 1$	q1: $n \leftarrow n + 1$	0
(end)	q1: $n \leftarrow n + 1$	1
(end)	(end)	2

Process p	Process q	n
p1: $n \leftarrow n + 1$	q1: $n \leftarrow n + 1$	0
p1: $n \leftarrow n + 1$	(end)	1
(end)	(end)	2

Algorithm 2.4: Assignment statements with one global reference

integer $n \leftarrow 0$

p

q

integer temp

integer temp

p1: $\text{temp} \leftarrow n$

q1: $\text{temp} \leftarrow n$

p2: $n \leftarrow \text{temp} + 1$

q2: $n \leftarrow \text{temp} + 1$

Process p	Process q	n	p.temp	q.temp
p1: temp←n	q1: temp←n	0	?	?
p2: n←temp+1	q1: temp←n	0	0	?
(end)	q1: temp←n	1	0	?
(end)	q2: n←temp+1	1	0	1
(end)	(end)	2	0	1

Escenari correcte

Process p	Process q	n	p.temp	q.temp
p1: temp←n	q1: temp←n	0	?	?
p2: n←temp+1	q1: temp←n	0	0	?
p2: n←temp+1	q2: n←temp+1	0	0	0
(end)	q2: n←temp+1	1	0	0
(end)	(end)	1	0	0

Escenari incorrecte

Restricció de *referència crítica limitada LCR*

- **Definició:** Una variable és una *referència crítica* si:
 - (a) Se li assigna un valor en un procés i també apareix en un altre
 - (b) o Si apareix en una expressió en un procés i té una assignació en un altre

Restricció de *referència crítica limitada LCR*

$$n \leftarrow n + 1$$

- n s'assigna dins p i apareix 2 vegades dins q :
Referència crítica per (a)
- $n + 1$ és crítica per (b) per que apareix en una expressió a p i és assignat a q

Variables volàtils

Algorithm 2.8: Volatile variables

integer $n \leftarrow 0$

p

integer local1, local2

p1: $n \leftarrow \text{some expression}$

p2: *computation not using n*

p3: $\text{local1} \leftarrow (n + 5) * 7$

p4: $\text{local2} \leftarrow n + 5$

p5: $n \leftarrow \text{local1} * \text{local2}$

q

integer local

q1: $\text{local} \leftarrow n + 6$

q2:

q3:

q4:

q5:

- Per l'intercalat el procés q pot ocupar qualsevol lloc en l'execució de p . Per efectes d'optimització durant la compilació n pot no tenir el valor més recent
- Volàtil: El compilador carrega i emmagatzema la variable cada cop que apareix

Situació de competició

Race condition

- Descriu l'error que es produeix en programes quan no han estat dissenyats adequadament per a la seva execució simultània amb altres.
- Un exemple típic és l'interbloqueig que es produeix quan dos processos estan esperant que l'altre faci una acció. Com que els dos estan esperant, cap arriba a realitzar l'acció que l'altre espera.

Counter.java

```
public class Counter implements Runnable {
//public class Counter extends Thread {
    static final int THREADS = 2;
    static final int MAX_COUNT = 10000000;
//    static volatile int n = 0;
    static int n = 0; // El compilador manté una còpia local a cada fil
    int id;

    public Counter(int id) {
        this.id = id;
    }

    @Override
    public void run() {
        int max = MAX_COUNT/THREADS;
        System.out.printf("Thread %d\n", id);
        for (int i=0; i < max; i++) {
            n += 1;
        }
    }

    public static void main(String[] args) throws InterruptedException {
        Thread[] threads = new Thread[THREADS];
        int i;

        for (i=0; i< THREADS; i++) {
            threads[i] = new Thread(new Counter(i));
            threads[i].start();
        }

        for (i=0; i< THREADS; i++) {
            threads[i].join();
        }
        System.out.printf("Counter value: %d Expected: %d\n", n, MAX_COUNT);
    }
}
```

El thread com un classe

```

import threading

MAX_COUNT = 10000000
THREADS = 2
n = 0

class CountThread(threading.Thread):
    def __init__(self):
        threading.Thread.__init__(self)
    def run(self):
        global n
        print("Thread {}".format(threading.current_thread().name))
        for i in range (MAX_COUNT/THREADS):
            n += 1

def main():
    threads = []
    for i in range(THREADS):
        # Create new threads
        t = CountThread()
        threads.append(t)
        t.start() # start the thread
    # Wait for all threads to complete
    for t in threads:
        t.join()
    print("Counter value: {} Expected: {}\n".format(n, MAX_COUNT))

if __name__ == "__main__":
    main()

```

El thread com la cridada a una funció

```

import threading

THREADS = 2
MAX_COUNT = 10000000
n = 0

def count():
    global n
    print("Thread {}".format(threading.current_thread().name))
    for i in range(MAX_COUNT/THREADS):
        n += 1

def main():
    threads = []
    for i in range(THREADS):
        # Create new threads
        t = threading.Thread(target=count)
        threads.append(t)
        t.start() # start the thread
    # Wait for all threads to complete
    for t in threads:
        t.join()
    print("Counter value: {} Expected: {}\n".format(n, MAX_COUNT))

if __name__ == "__main__":
    main()

```

Counter.c

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUM_THREADS 2
#define MAX_COUNT 10000000

// Just used to send the index of the id
struct tdata {
    int tid;
};

volatile int counter = 0;

void *count(void *ptr) {
    long i, max = MAX_COUNT/NUM_THREADS;
    int tid = ((struct tdata *) ptr)->tid;
    for (i=0; i < max; i++) {
        counter += 1; // The global variable, i.e. the critical section
    }
    printf("End %d counter: %d\n", tid, counter);
    pthread_exit(NULL);
}

int main (int argc, char *argv[]) {
    pthread_t threads[NUM_THREADS];
    int rc, i;
    struct tdata id[NUM_THREADS];

    for(i=0; i<NUM_THREADS; i++){
        id[i].tid = i;
        rc = pthread_create(&threads[i], NULL, count, (void *) &id[i]);
    }

    for(i=0; i<NUM_THREADS; i++){
        pthread_join(threads[i], NULL);
    }

    printf("Counter value: %d Expected: %d\n", counter, MAX_COUNT);
    pthread_exit(NULL);
}
```

counter.go

```
package main

import (
    "fmt"
    "runtime"
)

const (
    Procs      = 2
    MaxCount   = 10000000
    Goroutines = 2
)

var counter = 0

func run(id, counts int, done chan bool) {
    for i := 0; i < counts; i++ {
        counter++
    }
    fmt.Printf("End %d counter: %d\n", id, counter)
    done <- true
}

func main() {
    runtime.GOMAXPROCS(Procs)
    done := make(chan bool, 1)
    for i := 0; i < Goroutines; i++ {
        go run(i, MaxCount/Goroutines, done)
    }
    for i := 0; i < Goroutines; i++ {
        <-done
    }
    fmt.Printf("Counter value: %d Expected: %d\n", counter, MaxCount)
}
```

Counter.Ada

```
with Ada.Text_IO; use Ada.Text_IO;

procedure Counter is
  n: Integer := 0;
  pragma Volatile(n);
  task type Tasca_Comptador;
  task body Tasca_Comptador is
    temp: integer;
  begin
    for i in 1..5000000 loop
      temp := n;
      n := temp + 1;
    end loop;
    Put_Line("----->> n és " & Integer'Image(N));
  end Tasca_Comptador;

begin
  declare
    p, q : Tasca_Comptador;
  begin
    null;
  end;
  Put_Line("El valor de n és " & Integer'Image(N));
end Counter;
```